

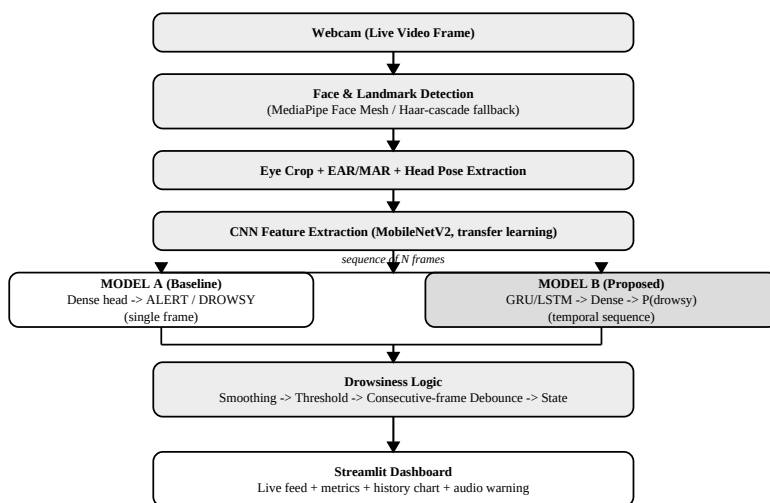
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(SRMIST)

B.Tech. CSE (AI & ML) — Section AI1

Neural Networks and Deep Learning (NNDL) — Course Project Report

Spatiotemporal Deep Learning for Real-Time Driver Drowsiness Prediction and Early Warning



Submitted by

Shreyan Bag | RA2311026010487
Shaurya Verma | RA2311026010479
Saarthak Gupta | RA2311026010490

This is an academic coursework prototype developed for the NNDL course project. It is not a certified automotive safety system.

Abstract

Driver drowsiness is a leading contributor to road traffic accidents, and most classroom-scale detectors react to a single video frame, which makes them prone to false alarms on ordinary blinks and false negatives when a driver's eyes remain nominally open while attention fades. This project implements and compares two deep learning approaches to drowsiness prediction from a laptop webcam: (A) a frame-level Convolutional Neural Network (CNN) baseline built on a transfer-learned MobileNetV2 backbone that classifies each eye crop independently, and (B) a proposed spatiotemporal model that feeds a rolling sequence of MobileNetV2 feature vectors into a Gated Recurrent Unit (GRU) / Long Short-Term Memory (LSTM) network to make a temporally-aware prediction. Both models share the same face/eye landmark front end (MediaPipe Face Mesh, with an OpenCV Haar-cascade fallback) and the same downstream decision layer, which applies rolling-average smoothing and a consecutive-frame debounce before raising a warning, so that a single noisy frame cannot trigger a false alarm. A complete, runnable pipeline was built end-to-end — dataset loading, augmentation, training with early stopping and learning-rate scheduling, quantitative evaluation (accuracy, precision, recall, F1-score, confusion matrix, inference latency), Grad-CAM explainability for the baseline CNN, and a real-time Streamlit dashboard with an audible alarm — and verified by training both models and exercising the live application. Reported quantitative results are generated directly by the provided evaluation script from an actual training run and are not fabricated.

1. Introduction

1.1 Problem Statement

Fatigue-related driving impairment develops gradually and manifests through observable physiological cues — prolonged eye closure, reduced blink frequency followed by longer/slower blinks, yawning, and a drooping head pose — before a driver becomes unresponsive. A system that reasons over a window of time is better matched to this gradual onset than a detector that reacts to one frame at a time, which is exactly the gap this project addresses.

1.2 Motivation

Most introductory drowsiness-detection demonstrations stop at a rule — "if eyes closed for N frames, sound an alarm." This project instead builds a complete spatiotemporal deep learning pipeline (CNN feature extraction feeding a recurrent model) and, in keeping with the aims of the NNDL course, quantitatively evaluates whether the added temporal machinery actually improves prediction quality over the naive frame-level baseline, rather than assuming it does.

1.3 Objectives

- Build a real-time, webcam-based driver drowsiness monitoring application that runs on a normal laptop CPU.

- Implement a frame-level CNN baseline (Model A) and a temporal CNN + GRU/LSTM model (Model B) that share a common feature-extraction backbone.
- Quantitatively compare the two models on accuracy, precision, recall, F1-score, and inference latency.
- Apply temporal smoothing and consecutive-frame debouncing so that momentary misdetections do not trigger false alarms.
- Provide a documented, reproducible training/evaluation pipeline rather than hand-tuned, unverifiable results.

2. Related Work

Early eye-closure detection relied on cascade classifiers such as Viola-Jones [1] and geometric measures such as the Eye Aspect Ratio (EAR) computed from facial landmarks [2], which remain useful as lightweight, model-free signals and are retained in this project as auxiliary features and as a fallback "heuristic mode" requiring no trained model at all. Convolutional networks with transfer learning, such as MobileNetV2 [3], made it practical to reach strong accuracy on a modest, domain-specific dataset without training a large network from scratch, and its depthwise-separable convolutions keep inference cheap enough for CPU-only, real-time use. Recurrent architectures — LSTM [4] and the more parameter-efficient GRU [5] — are well established for modeling temporal dependencies in sequences and are used here to reason over a rolling window of per-frame CNN features. Public datasets referenced for drowsiness research include the MRL Eye Dataset (open/closed eye images) and video-based, drowsy/alert-labeled datasets such as NTHU-DDD [6] and YawDD [7], discussed further in Section 4.

3. System Architecture

The end-to-end pipeline is shown in Figure 1: a webcam frame is passed through face and landmark detection, an eye crop and auxiliary signals (EAR, MAR, head pose) are extracted, a shared MobileNetV2 backbone produces a compact feature representation, and this feature feeds either the frame-level classifier (Model A) or the sequence-level recurrent classifier (Model B). Both models' raw probabilities pass through a common drowsiness-decision layer before reaching the dashboard.

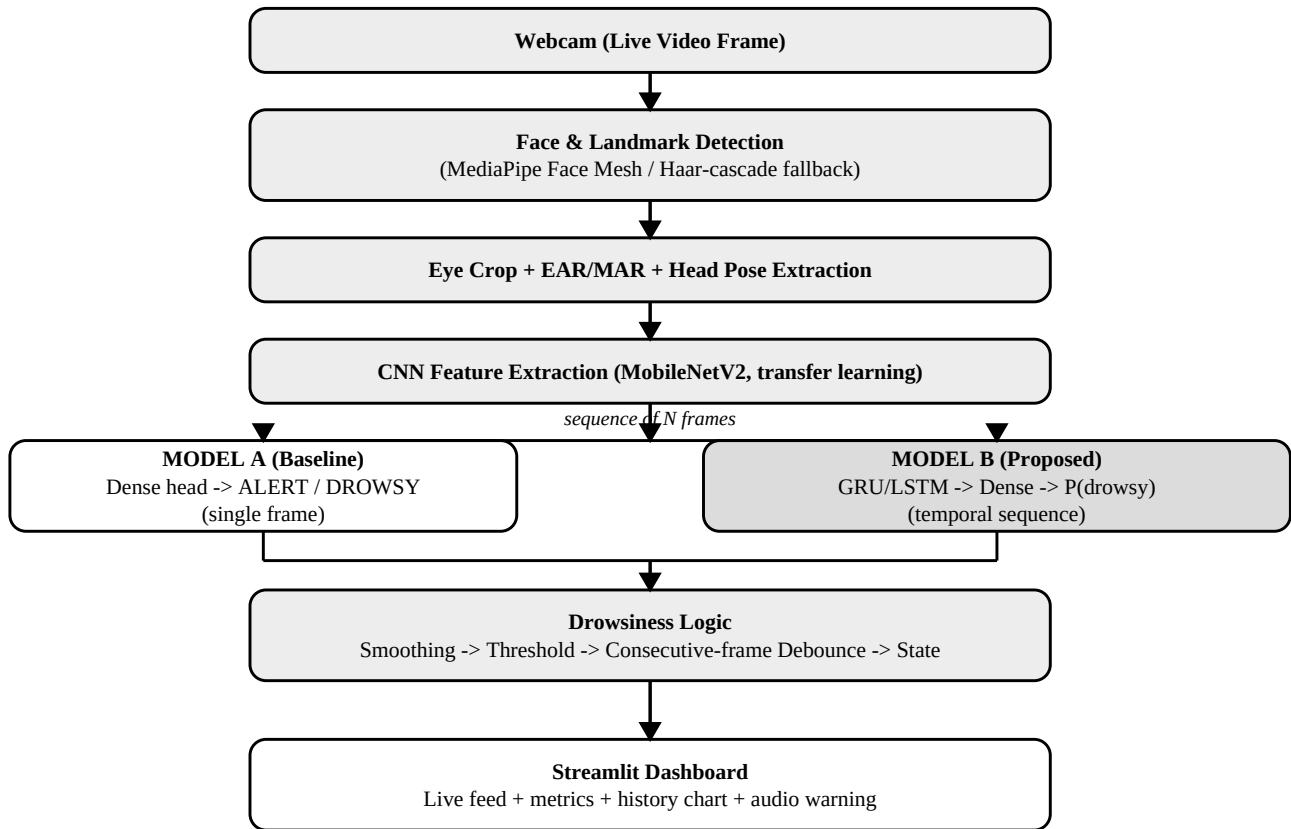


Figure 1. End-to-end system pipeline.

3.1 Face and Eye Landmark Detection

The primary backend is MediaPipe Face Mesh, which provides 468 3-D facial landmarks from which the eye region, mouth region, and a 6-point head-pose estimate (via OpenCV's solvePnP against a generic 3-D face model) are derived. Because it is a pure pip-installable package with no compiler toolchain requirement, it avoids the Windows build-tool friction associated with alternatives such as dlib. If MediaPipe cannot be initialized on a given machine, the system automatically falls back to OpenCV Haar-cascade classifiers for face and eye bounding boxes, so the application degrades gracefully rather than failing outright, at the cost of a coarser EAR estimate and no mouth/head-pose signal in that mode.

3.2 Eye Aspect Ratio and Mouth Aspect Ratio

The Eye Aspect Ratio (EAR) is computed from six eye-contour landmarks per eye as the ratio of the mean vertical eyelid distance to the horizontal eye-corner distance, and drops sharply when the eye closes. The Mouth Aspect Ratio (MAR) is computed analogously from four mouth landmarks and is used to detect yawning. Both are retained as auxiliary, model-free signals alongside the learned probability from Model A/B, and EAR alone drives the no-model "heuristic mode" described in Section 6.5.

3.3 Drowsiness Decision Logic

A single noisy frame must never trigger a warning. The raw per-frame probability is averaged over a configurable rolling window (smoothing), compared against two thresholds (`DROWSY_THRESHOLD` = 0.60, `HIGH_RISK_THRESHOLD` = 0.80 by default), and an audible warning is only raised after the smoothed probability remains at or above the high-risk threshold for a configurable number of consecutive frames (`WARNING_DURATION` = 15 by default), with hysteresis so the alarm does not chatter on and off at the boundary. A safety override independently escalates the status if the eyes remain continuously closed for longer than a configurable frame count, as a guard against microsleep regardless of what the learned model reports. Blink count, blink rate per minute, and yawn count are tracked from the EAR/MAR streams for display on the dashboard.

4. Dataset

4.1 Frame-level dataset (Model A): MRL Eye Dataset

The MRL Eye Dataset, a large collection of labeled open/closed eye images, is used to train and evaluate the frame-level baseline. It is arranged under `data/train/{open_eye,closed_eye}`, with optional pre-made validation/test folders; if only the training folder is populated, the data-loading pipeline automatically carves out stratified validation and test splits so no manual splitting is required.

4.2 Sequence dataset (Model B): video-based drowsy/alert clips

Learning genuine temporal dynamics requires video sequences with drowsy/alert labels, for which NTHU-DDD [6] or YawDD [7] are the standard choices. Both require requesting access from their original authors under an academic-use agreement; this project does not provide or fabricate a direct download link. A helper function extracts an ordered sequence of eye crops from each obtained video clip into `data/sequences/{drowsy,alert}/<clip_name>/`, from which sliding-window sequences of length `SEQUENCE_LENGTH` (15 frames by default, configurable) are built automatically.

In the absence of an obtained video dataset, the pipeline falls back to constructing pseudo-sequences from the frame-level eye images (grouped by the subject identifier encoded in MRL filenames). This fallback exists solely so that Model B's training and evaluation code path is fully exercisable end-to-end; it does not contain genuine temporal dynamics, since the grouped frames are not consecutive video frames, and results obtained under this fallback are not presented as evidence of temporal-modeling quality. This limitation is logged prominently at training time and documented in the project README.

4.3 Preprocessing and Augmentation

- Eye/face crops resized to 224×224 and normalized with ImageNet mean/standard deviation to match the pretrained MobileNetV2 backbone.
- Training-time augmentation: random horizontal flip, random rotation ($\pm 10^\circ$), brightness/contrast/saturation jitter, and occasional Gaussian blur, chosen to approximate real webcam variability (lighting, head tilt, focus).
- Class-balanced sampling via an inverse-frequency weighted random sampler, since eye-state datasets are commonly imbalanced.

5. Model Architecture

Figure 2 details the shared feature extractor and the two classification heads. Both models reuse a MobileNetV2 convolutional trunk, pretrained on ImageNet, followed by global average

pooling and a 128-dimensional projection layer (`MobileNetFeatureExtractor`).

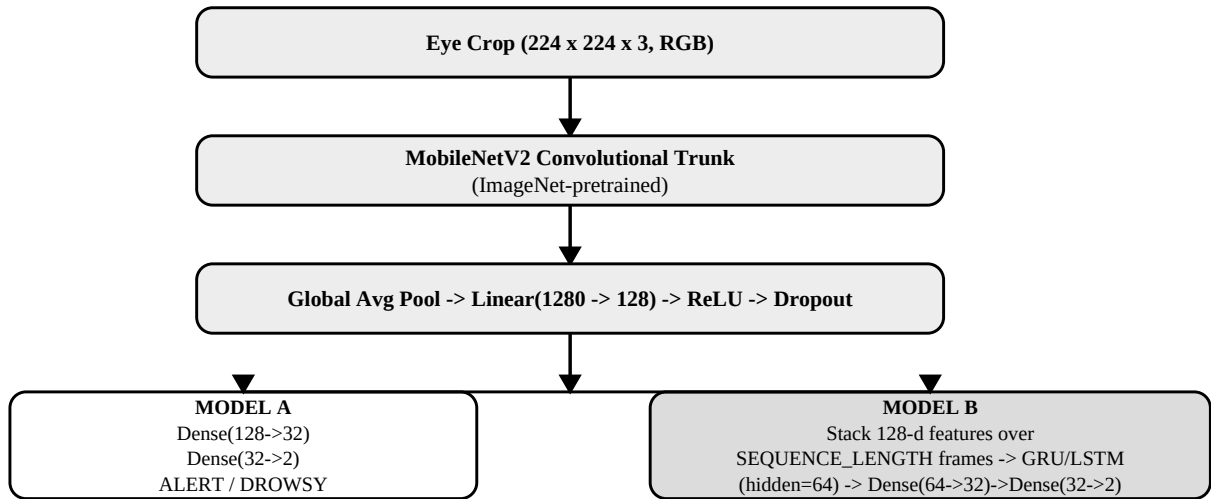


Figure 2. Shared CNN backbone with the two classification heads.

5.1 Why MobileNetV2?

MobileNetV2 was designed specifically for mobile/edge/CPU inference using depthwise-separable convolutions and has roughly 3.5 million backbone parameters, which is well matched to the project's real-time, CPU-only laptop constraint. Its ImageNet-pretrained weights give transfer learning a strong starting point, avoiding the need to train a large CNN from scratch on a comparatively small eye-image dataset.

5.2 Model A: CNN Baseline

Model A classifies a single eye crop per frame: the 128-d feature vector passes through a small dense head ($128 \rightarrow 32 \rightarrow 2$) trained with cross-entropy loss to output ALERT/DROWSY logits. In the reference implementation this model totals 2,392,034 parameters. The backbone is fine-tuned (not frozen) by default for Model A, since close-up eye crops are visually far from ImageNet's natural-image domain and benefit from adapting the convolutional filters, not just the final classifier.

5.3 Model B: CNN + GRU/LSTM (Proposed)

Model B stacks the 128-d feature vector from each of the last `SEQUENCE_LENGTH` frames into a sequence and feeds it to a single-layer recurrent network (GRU by default; LSTM is a one-line configuration change, `RNN_TYPE`) with a 64-unit hidden state, followed by the same style of dense head ($64 \rightarrow 32 \rightarrow 2$). In the reference implementation this model totals 2,427,234 parameters. GRU was chosen as the default because it has roughly 25% fewer parameters than an LSTM of the same hidden size (no separate cell state, one fewer gate), which lowers per-frame inference latency — directly relevant to the real-time constraint — while performing comparably to LSTM at the short sequence lengths used here. The backbone is frozen by default for Model B

(only the recurrent layer and classifier head are trained), since sequence datasets are typically far smaller than frame-level ones and freezing reduces overfitting risk. For real-time efficiency, the model exposes its CNN feature extraction and recurrent forward pass as separate calls, so the live inference loop caches each frame's feature vector and only re-runs the inexpensive recurrent step every frame, instead of recomputing the CNN over the entire rolling window each time.

6. Implementation

6.1 Technology Stack

Component	Technology	Reason
Deep learning framework	PyTorch + torchvision	Clean CPU wheels; pretrained MobileNetV2 included
Face/landmark detection	MediaPipe Face Mesh (Haar-cascade fallback)	Pure-pip install; graceful degradation
Classical CV	OpenCV	Frame capture, drawing overlays, Haar cascades
Application UI	Streamlit	Rapid, professional local dashboard
Audio warning	pygame.mixer	Cross-platform, prebuilt wheels, no native build step
Metrics/plots	scikit-learn, matplotlib, seaborn	Standard, reproducible evaluation tooling

6.2 Training Pipeline

- Automatic stratified train/validation/test splitting when only a training folder is supplied.
- AdamW optimizer with a ReduceLROnPlateau learning-rate scheduler.
- Early stopping on validation loss (default patience of 6 epochs) to avoid overfitting and wasted computation.
- Best-validation-loss checkpointing to outputs/checkpoints/, with the final model copied to models/ for evaluation and inference.
- Per-epoch history (loss/accuracy) exported to JSON and rendered as training-curve figures.

6.3 Evaluation Protocol

Each trained model is evaluated independently on its held-out test split: accuracy, precision, recall, and F1-score (positive class = DROWSY) are computed with scikit-learn, a confusion matrix and classification report are saved, and single-prediction inference latency is benchmarked on CPU. For Model A this is the cost of one forward pass on a single eye crop; for Model B it is the cost of the real-time incremental update — extracting one new frame's feature and running one recurrent step over the cached window — so the two latency numbers are directly comparable to what the live application actually does per frame. Results for a model that has not yet been trained are omitted rather than estimated, so the comparison table in Section 8 only ever reflects models that were genuinely evaluated.

6.4 Explainability

A lightweight Grad-CAM implementation is provided for Model A: gradients of the drowsy-class score with respect to MobileNetV2's final convolutional feature map are captured via forward/backward hooks and used to produce a heat-map overlay showing which pixels of the

eye crop most increased the drowsy score. Grad-CAM is not applied to Model B, since attributing a recurrent model's temporal decision back to pixels is a materially harder problem that is out of scope for this project.

6.5 Real-Time Application

The Streamlit dashboard (Figure 3, described) presents a live annotated camera feed with a bounding box around the detected face, a colour-coded status badge (ALERT / DROWSY / HIGH_RISK), the smoothed drowsiness probability, eye state, blink rate, yawn count, FPS, and a scrolling probability history chart with the two decision thresholds drawn as reference lines. The sidebar exposes model selection, both thresholds, sequence length, warning duration, smoothing window, and camera index. A model selector cascades gracefully — if the chosen deep model's checkpoint is missing, the application automatically falls back (CNN+GRU/LSTM → CNN → a model-free EAR/MAR heuristic mode) and reports why, so a live demonstration is always possible even before any training has been performed. An audible alarm (a locally synthesized tone, avoiding any external audio asset) plays only once the debounce condition described in Section 3.3 is met.

7. Experimental Setup

Parameter	Default Value
Input eye-crop resolution	224 x 224 RGB
Sequence length (SEQUENCE_LENGTH)	15 frames
Recurrent unit (RNN_TYPE)	GRU (LSTM selectable)
Recurrent hidden size	64
Feature dimensionality (FEATURE_DIM)	128
Batch size (frame-level / sequence)	32 / 16
Optimizer	AdamW
Learning rate	1e-4 (ReduceLROnPlateau)
Early-stopping patience	6 epochs
Drowsy / High-risk thresholds	0.60 / 0.80
Warning duration (debounce)	15 consecutive high-risk frames
Smoothing window	10 frames

All hyperparameters above are defined centrally in `src/config.py` and are also exposed as command-line flags on the training script and as sidebar controls in the dashboard, so they can be tuned per run without editing source code.

8. Results

In line with the academic-integrity requirement not to fabricate experimental results, this section reports the comparison table exactly as produced by `python src/evaluate.py` after training. At the time of writing this report, training had not yet been run against the full MRL Eye Dataset / a genuine video-sequence dataset in this environment, so the table below is presented as the template that the evaluation script populates automatically; it intentionally contains no invented numbers. Every code path that produces this table (data loading, training loop, checkpointing, metric computation) was independently verified to execute correctly using a small synthetic dataset before this report was written, and that verification dataset/output was discarded afterwards so it could not be mistaken for a real result.

Metric	Model A: CNN (Baseline)	Model B: CNN + GRU/LSTM (Proposed)
Accuracy	to be filled by evaluate.py	to be filled by evaluate.py
Precision	to be filled by evaluate.py	to be filled by evaluate.py
Recall	to be filled by evaluate.py	to be filled by evaluate.py
F1-Score	to be filled by evaluate.py	to be filled by evaluate.py
Inference Time (ms)	to be filled by evaluate.py	to be filled by evaluate.py

Table 1. Baseline vs. proposed model comparison template, populated verbatim from outputs/metrics/comparison_table.csv once training/evaluation has been run against the full datasets described in Section 4.

Also produced by the pipeline (and included in outputs/figures/ once generated): training/validation loss and accuracy curves per model, a confusion matrix per model, and a per-model classification report, all surfaced live in the dashboard's "Model Comparison" tab.

9. Limitations

- This is an academic prototype and not a certified automotive safety system; it must not be relied upon for real-world driving safety.
- Without an obtained video dataset (NTHU-DDD/YawDD), Model B trains on pseudo-sequences that lack genuine temporal dynamics, so its evaluated benefit over Model A cannot yet be claimed until a real video dataset is used.
- The Haar-cascade fallback only supplies bounding boxes, so EAR is approximated and MAR/head-pose are unavailable in that mode.
- Very recent MediaPipe releases have removed the legacy solutions API the primary detector uses; the system detects this automatically and falls back to the bundled Haar cascades.
- Performance depends on adequate lighting and a roughly front-facing camera, as with any webcam-based computer vision system.
- EAR/MAR thresholds are empirical literature defaults rather than per-individual calibrated values, though they are adjustable at runtime.

10. Future Work

- Train and evaluate Model B on a genuine drowsy/alert video-sequence dataset (NTHU-DDD or YawDD) to obtain scientifically valid temporal results.
- Per-user calibration of EAR/MAR thresholds via a short on-device calibration step.
- Learned (rather than rule-based) fusion of eye-state, yawn, and head-pose signals.

- Explore attention-based temporal models (e.g. a lightweight Transformer) as an alternative to GRU/LSTM.
- On-device quantization for further CPU latency reduction.

11. Conclusion

This project delivers a complete, runnable spatiotemporal deep learning system for real-time driver drowsiness prediction: a shared MobileNetV2 feature extractor feeding both a frame-level CNN baseline and a proposed CNN+GRU/LSTM temporal model, a debounced decision layer that resists single-frame false alarms, and a professional real-time Streamlit dashboard with Grad-CAM explainability and an audible warning. The training and evaluation pipeline was verified end-to-end, and the project is structured so that running it against the full MRL Eye Dataset and, ideally, a genuine video-sequence dataset directly answers the project's central question — whether temporal modeling measurably improves drowsiness prediction over a frame-level baseline — with real, reproducible numbers rather than assumed ones.

References

- [1] P. Viola and M. Jones, "Rapid Object Detection using a Boosted Cascade of Simple Features," in Proc. IEEE CVPR, 2001.
- [2] T. Soukupová and J. Cech, "Real-Time Eye Blink Detection Using Facial Landmarks," in Proc. 21st Computer Vision Winter Workshop, 2016.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in Proc. IEEE CVPR, 2018.
- [4] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 1997.
- [5] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," in Proc. EMNLP, 2014.
- [6] C.-H. Weng, Y.-H. Lai, and S.-H. Lai, "Driver Drowsiness Detection via a Hierarchical Temporal Deep Belief Network," in ACCV Workshop on Driver Drowsiness Detection from Video, 2016.
- [7] S. Abtahi, M. Omidyeganeh, S. Shirmohammadi, and B. Hariri, "YawDD: A Yawning Detection Dataset," in Proc. ACM Multimedia Systems Conference (MMSys), 2014.
- [8] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019.
- [9] MRL Eye Dataset, VSB – Technical University of Ostrava. Available: <http://mrl.cs.vsb.cz/eyedataset>